

Network Security

BITS Pilani

Subhadip Singha
Computer Science & Information System



innovate achieve lead

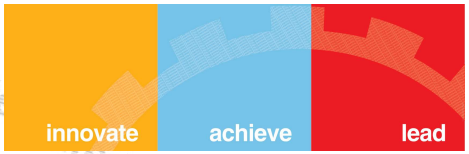
BITS Pilani

ES ZG513/ SS ZG513, Network Security
Lecture No V.

Random Numbers

- many uses of **random numbers** in cryptography
  - nonces in authentication protocols to prevent replay
  - session keys
  - public key generation
  - keystream for a one-time pad
- in all cases its critical that these values be
  - statistically random, uniform distribution, independent
  - unpredictability of future values from previous values
- true random numbers provide this
- care needed with generated random numbers

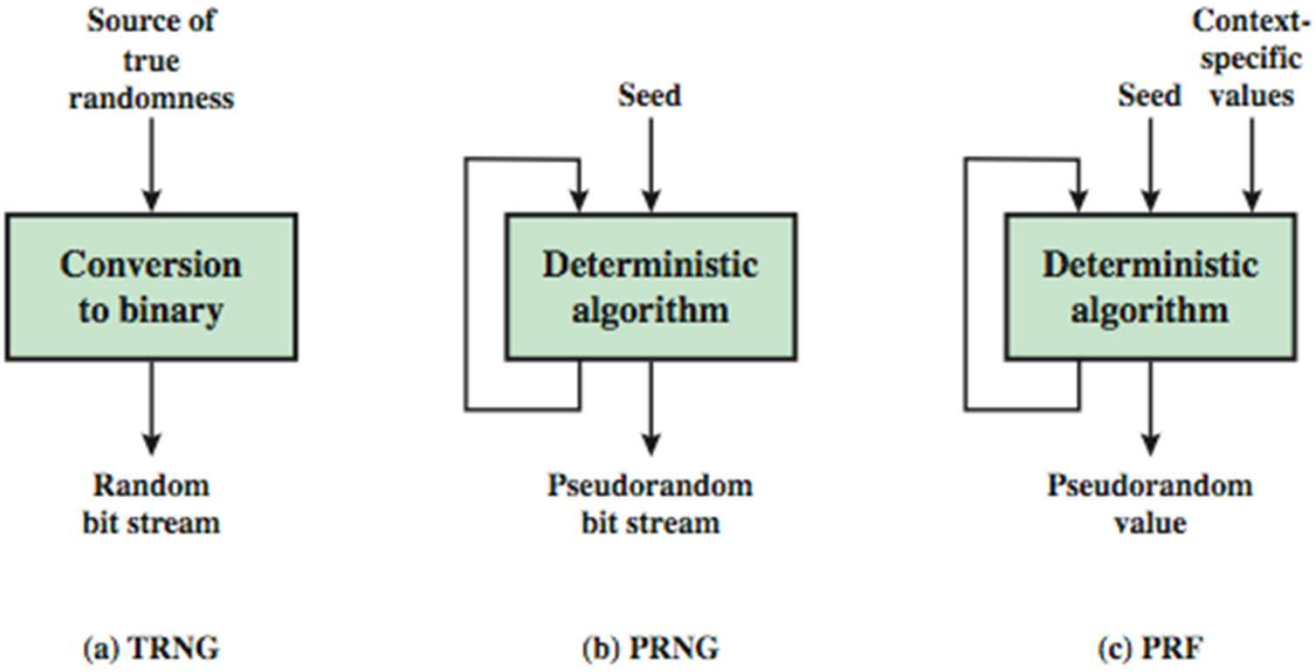
Pseudorandom Number Generators (PRNGs)



- often use deterministic algorithmic techniques to create "random numbers"
  - although are not truly random
  - can pass many tests of "randomness"
- known as "pseudorandom numbers"
- created by "Pseudorandom Number Generators (PRNGs)"

Work Integrated Learning Programmes

Random & Pseudorandom Number Generators



PRNG Requirements

- randomness
  - uniformity, scalability, consistency
- unpredictability
  - forward & backward unpredictability
  - use same tests to check
- characteristics of the seed
  - secure
  - if known adversary can determine output
  - so must be random or pseudorandom number

Work Integrated Learning Programmes

PRNG Requirements

innovate achieve lead

- Randomness
  - Uniformity – at any point in the generation of the PRN sequence, the occurrence of a zero or a one is equally likely (i.e., p = 0.5)
  - Scalability – any test for randomness applicable to a sequence can also be applied to any random subsequence (it should pass)
  - Consistency – the characteristics of the PRN sequence of the PRNG must not depend on the seed used

Work Integrated Learning Programmes

Linear Congruential Generator

innovate achieve lead

- common iterative technique using:

$X_{n+1} = (aX_n + c) \bmod m$

- given suitable values of parameters can produce a long random-like sequence
- suitable criteria to have are:
  - function generates a full-period
  - generated sequence should appear random
  - efficient implementation with 32-bit arithmetic
- note that an attacker can reconstruct sequence given a small number of values
- have possibilities for making this harder

Blum Blum Shub Generator

- based on public key algorithms
- use least significant bit from iterative equation:
  - $x_i = x_{i-1}^2 \bmod n$
  - where n=p.q, and primes p,q=3 mod 4
- unpredictable, passes **next-bit** test
- security rests on difficulty of factoring N
- is unpredictable given any run of bits
- slow, since very large numbers must be used
- too slow for cipher use, good for key generation

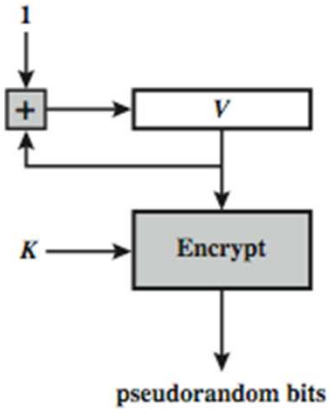
Using Block Ciphers as PRNGs

- for cryptographic applications, can use a block cipher to generate random numbers
- often for creating session keys from master key
- CTR

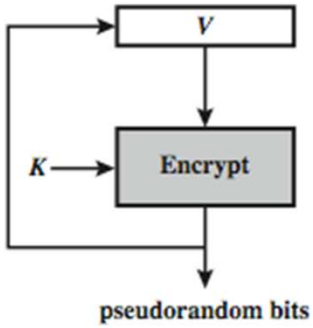
$$X_i = E_K[V_i]$$

- OFB

$$X_i = E_K[X_{i-1}]$$

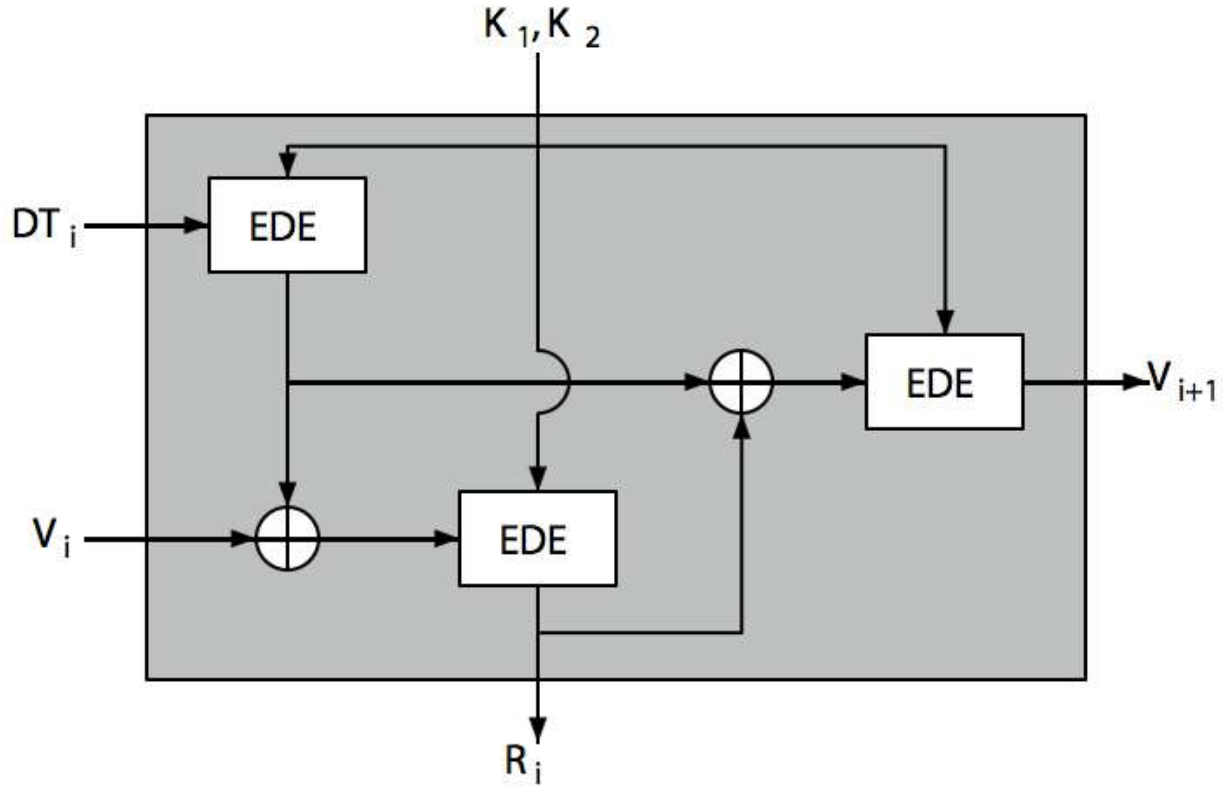


(a) CTR Mode



(b) OFB Mode

ANSI X9.17 PRG



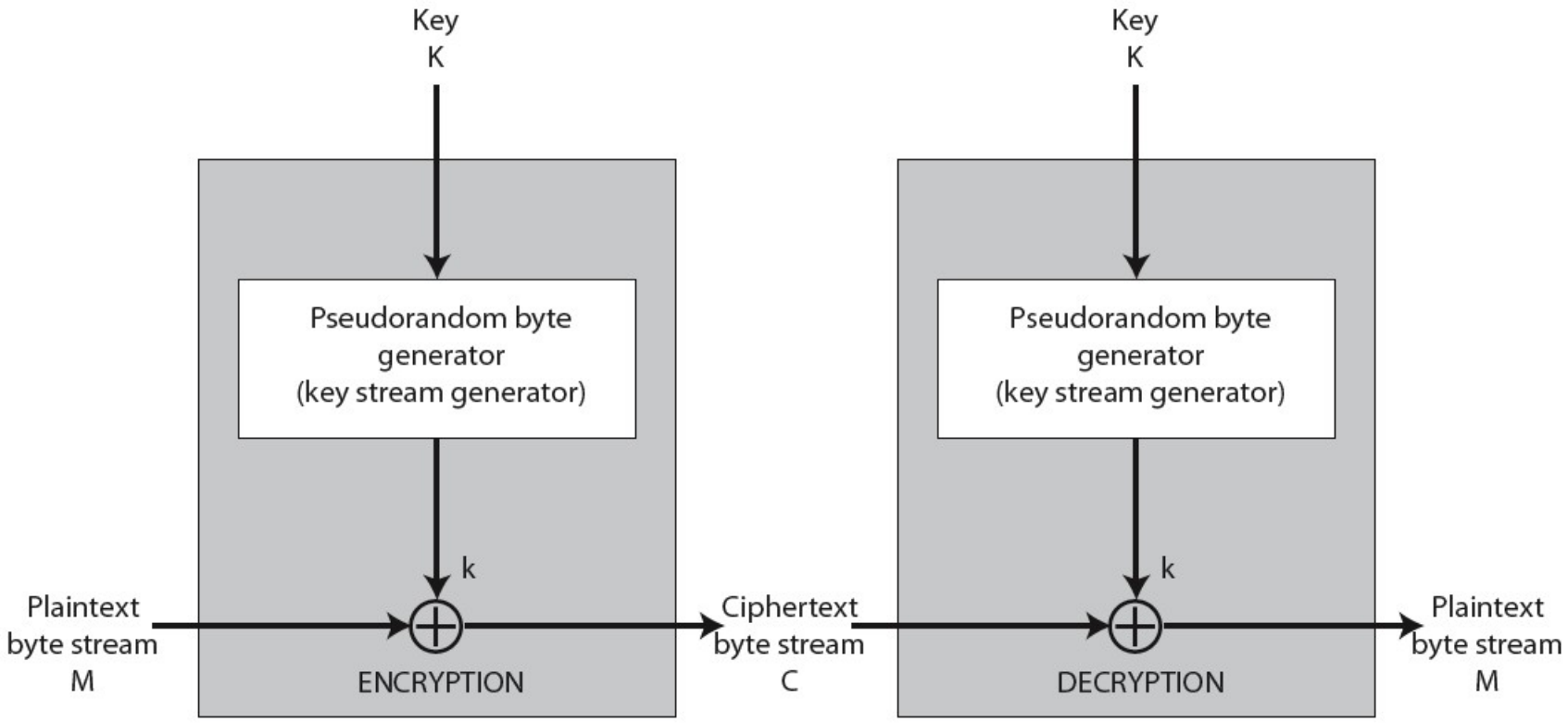
Dti = date and time

Stream Ciphers

innovate achieve lead

- process message bit by bit (as a stream)
- have a pseudo random **keystream**
- combined (XOR) with plaintext bit by bit
- randomness of **stream key** completely destroys statistically properties in message
  - $C_i = M_i \text{ XOR } StreamKey_i$
- but must never reuse stream key
  - otherwise can recover messages (cf book cipher)

Stream Cipher Structure



Stream Cipher Properties

- some design considerations are:
  - long period with no repetitions
  - statistically random
  - depends on large enough key
  - large linear complexity
- properly designed, can be as secure as a block cipher with same size key
- but usually simpler & faster

Work Integrated Learning Programmes

RC4

- a proprietary cipher owned by RSA DSI
- another Ron Rivest design, simple but effective
- variable key size, byte-oriented stream cipher
- widely used (web SSL/TLS, wireless WEP/WPA)
- key forms random permutation of all 8-bit values
- uses that permutation to scramble input info processed a byte at a time

RC4 Key Schedule

- starts with an array S of numbers: 0..255
- use key to well and truly shuffle
- S forms **internal state** of the cipher

```
for i = 0 to 255 do
S[i] = i  // init S to identity
T[i] = K[i mod keylen]) // extend key
j = 0                  // location of swap
for i = 0 to 255 do // for each element
                    // update location of swap
j = (j + S[i] + T[i]) (mod 256)
swap (S[i], S[j])   // permute
```

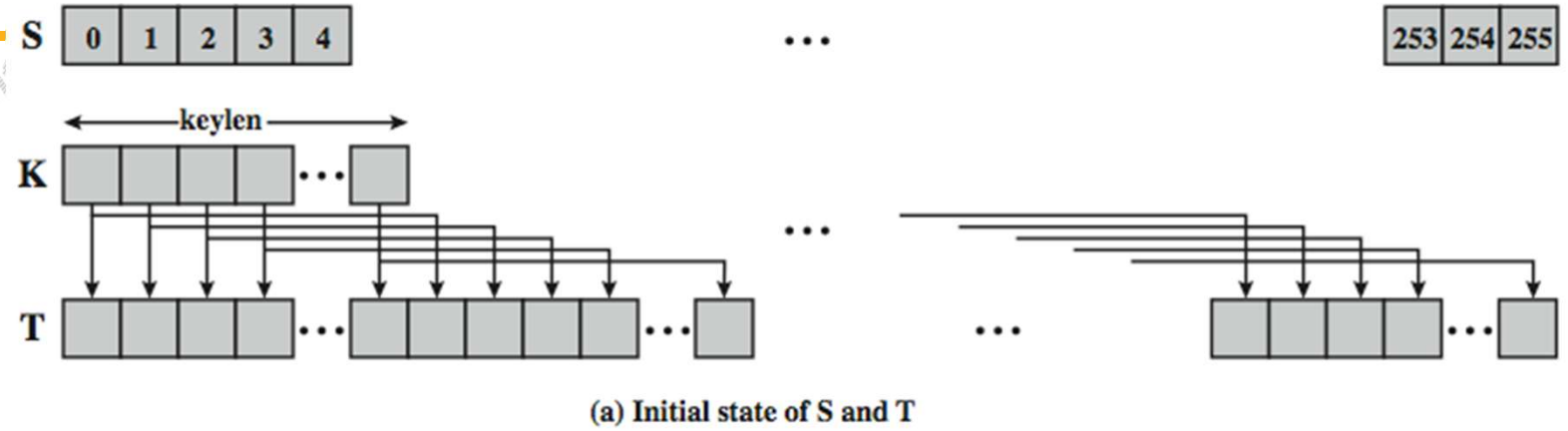
RC4 Encryption

- encryption continues shuffling array values
- sum of shuffled pair selects "stream key" value from permutation
- XOR S[t] with next byte of message to en/decrypt

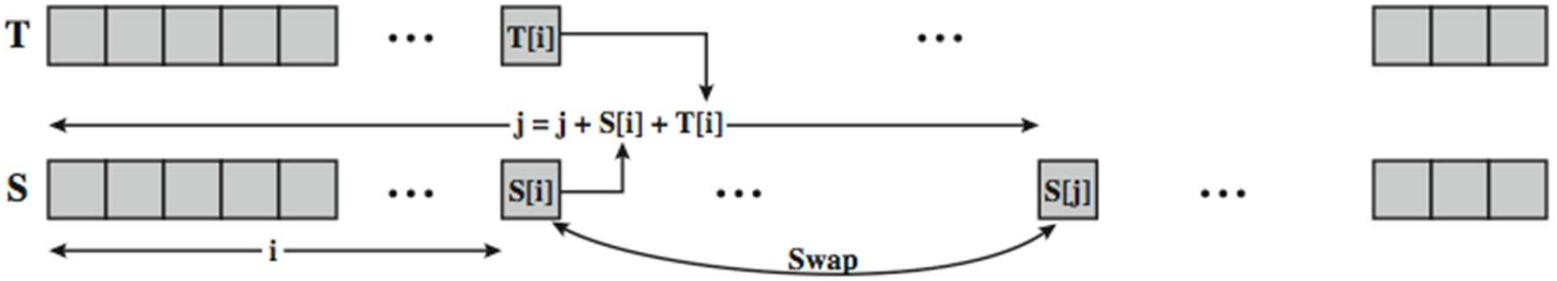
```
i = j = 0
for each message byte Mi
i = (i + 1) (mod 256)   // rotate thru S
j = (j + S[i]) (mod 256)   // update swap location
swap(S[i], S[j])          // update permutation S
t = (S[i] + S[j]) (mod 256)   // pick key index
Ci = Mi XOR S[t]          // encrypt
```

initialize permutation

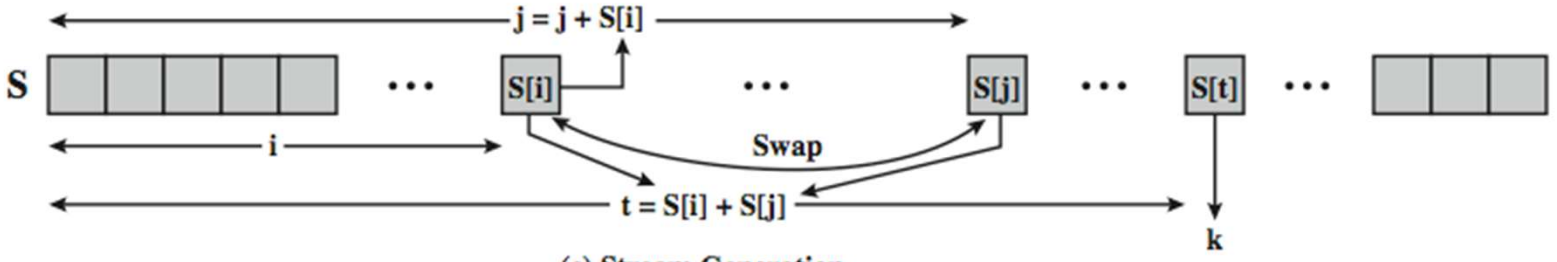
extend key



(a) Initial state of S and T



(b) Initial permutation of S

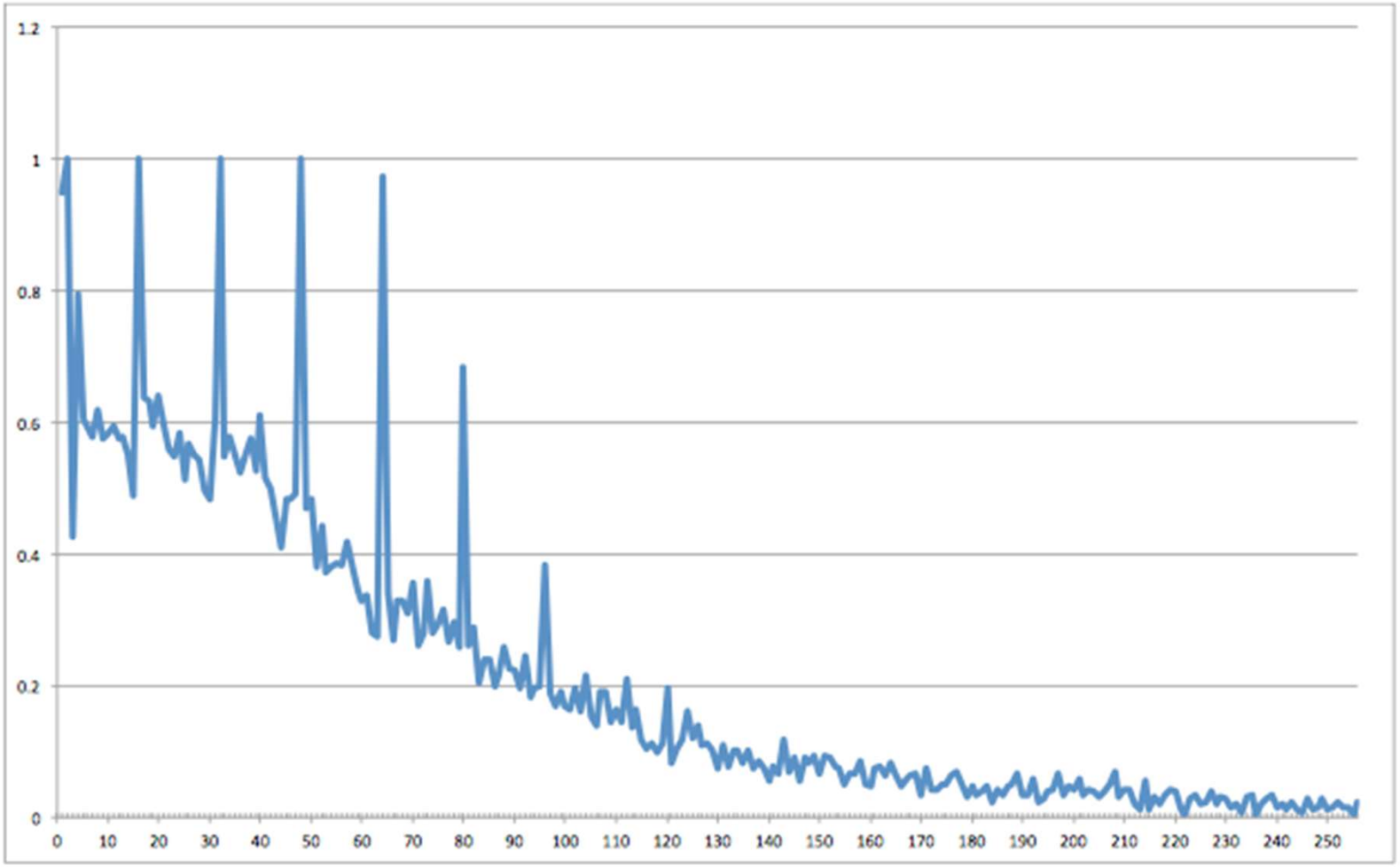


(c) Stream Generation

RC4 Security

- claimed secure against known attacks
  - However, have some analyses, some on the verge of practical
  - first 256 bytes have bias
  - multiple keys/same plaintext attack
- result is very non-linear
- since RC4 is a stream cipher, must **never reuse a key**
- have a concern with WEP, but due to key handling rather than RC4 itself

RC4 Security



Probability of recovery of plaintext from 2^{24} ciphertexts vs. byte position

Natural Random Noise

- best source is natural randomness in real world
- find a regular but random event and monitor
- do generally need special h/w to do this
  - eg. radiation counters, radio noise, audio noise, thermal noise in diodes, leaky capacitors, mercury discharge tubes etc
- starting to see such h/w in new CPUs
  - Intel Ivy Bridge, Via Padlock
- problems of **bias** or uneven distribution in signal
  - have to compensate for this when sample, often by passing bits through a hash function
  - best to only use a few noisiest bits from each sample
  - RFC4086 recommends using multiple sources + hash